Appendix: Extensions of the Augmented Join Graph

In the main paper (§3.1 and §3.3), Erupt focuses on SPJ queries with inner joins and deterministic UDF predicates. In this appendix, we discuss how the equivalence rules (Rules 1–3) and the augmented join graph can be extended to queries with non-deterministic UDF predicates (Appendix A) and non-inner joins (Appendix B).

A NON-DETERMINISTIC UDF PREDICATES

Equivalence Rules. In general UDF workloads (e.g., SQL-ProcBench [4] and UDFbench [2]), the few non-deterministic UDFs typically arise from introducing random logic (e.g., generating random numbers). If we relax the determinism assumption in §3.1 to allow such non-deterministic UDFs, we can refer to how mature database systems (e.g., Spark [3]) handle non-deterministic UDF predicates and analyze Rule 1 and Rule 3 as follows:

- **Rule 1 (UDF Pullup/Pushdown).** For a non-deterministic UDF predicate U_k with $Req(U_k) = \{T_1\}$, if a tuple in T_1 matches multiple tuples in T_2, $\sigma_{U_k}(T_1) \bowtie T_2$ invokes U_k only once on that tuple, whereas $\sigma_{U_k}(T_1 \bowtie T_2)$ invokes U_k multiple times on it, which may yield different outputs for the same tuple. Thus, Spark prohibits moving non-deterministic UDF predicates across join operators. Following this practice, Rule 1 still applies only to deterministic UDF predicates.
- **Rule 3 (UDF Reorder).** For two UDF predicates U_i and U_j with $Req(U_i) = \{T_1\}$ and $Req(U_j) = \{T_1\}$, assuming U_i is non-deterministic, in $\sigma_{U_j}\sigma_{U_i}(T_1)$, U_i is evaluated on all tuples of T_1, whereas in $\sigma_{U_i}\sigma_{U_j}(T_1)$, T_1 is filtered by U_j first before being fed into U_i, which changes the multiset of input tuples fed into U_i. Thus, Spark prohibits moving any predicate across a non-deterministic predicate. Following this practice, Rule 3 still applies only when both UDF predicates are deterministic.

Based on the above analysis, Rules 1–3 require no modification when a query contains non-deterministic UDF predicates. Moreover, for the initial relational algebra expression translated from the user query, the candidate plans generated by jointly applying these three rules ensure that the multiset of input tuples for each non-deterministic UDF remains unchanged. For example, given an initial relational algebra expression $Q_{\text{init}} = \sigma_{U_{\text{nd}}}(\sigma_{U_{\text{det}}}(T_1) \bowtie T_2) \bowtie (T_3 \bowtie T_4)$ with a deterministic UDF U_{det} and a non-deterministic UDF U_{nd} (where $Req(U_{\text{det}}) = Req(U_{\text{nd}}) = \{T_1\}$), applying Rules 1–3 can generate the candidate plan $\sigma_{U_{\text{nd}}}(\sigma_{U_{\text{det}}}(T_2 \bowtie T_1)) \bowtie (T_3 \bowtie T_4)$, where the multiset of input tuples fed into U_{nd} remains unchanged.

Graph Construction and Enumeration. The augmented join graph $H^+ = (V^+, E^+)$ can be extended to queries containing non-deterministic UDFs as follows. For graph construction, the existing process requires no modification: a non-deterministic UDF U_{nd} is likewise added as an individual node in V_U with its hyperedge defined as ($left = Req(U_{\text{nd}})$, $right = \{U_{\text{nd}}\}$). For csg-cmp-pair enumeration, we only need to add semantic constraints. Specifically, for a single non-deterministic UDF U_{nd}, let S_{in} denote the node set corresponding to its input subplan in the initial relational algebra expression, and $S_{\text{out}} = V^+ \setminus (S_{\text{in}} \cup \{U_{\text{nd}}\})$ denote the external node set. When enumerating a csg-cmp-pair (P_1, P_2), two constraints are enforced: (i) if $P_2 = \{U_{\text{nd}}\}$, we require $P_1 = S_{\text{in}}$; and (ii) if $U_{\text{nd}} \notin P_1 \cup P_2$, we require $P_1, P_2 \subseteq S_{\text{in}}$ or $P_1, P_2 \subseteq S_{\text{out}}$.

Taking the above Q_{init} as an example, we have $S_{\text{in}} = \{T_1, T_2, U_{\text{det}}\}$ and $S_{\text{out}} = \{T_3, T_4\}$: constraint (i) rules out pairs such as $(\{T_1\}, \{U_{\text{nd}}\})$, ensuring that U_{nd} is evaluated via $(\{T_1, T_2, U_{\text{det}}\}, \{U_{\text{nd}}\})$ only after S_{in} is fully merged; constraint (ii) rules out pairs such as $(\{T_2\}, \{T_3\})$, ensuring that nodes in S_{in} can be merged with subplans in S_{out} (e.g., $\{T_3\}$ or $\{T_3, T_4\}$) only after being merged with $\{U_{\text{nd}}\}$ into $\{T_1, T_2, U_{\text{det}}, U_{\text{nd}}\}$. In addition, when a query contains multiple non-deterministic UDFs, each introduces the above two constraints based on its own S_{in} and S_{out}, and an enumerated csg-cmp-pair (P_1, P_2) only needs to satisfy all these constraints.

Practical Remarks. Although the augmented join graph can be extended as above, non-deterministic UDFs are not the focus of our paper for two reasons: (i) they are rare in existing UDF benchmarks (2/24 in SQL-ProcBench and 1/24 in UDFbench); and (ii) following the practice of Spark only preserves the input tuple multiset of U_{nd}, which still cannot strictly guarantee semantic equivalence across candidate plans (e.g., operator reordering within S_{in} changes the tuple invocation order of U_{nd}, which may still alter query results).

B NON-INNER JOINS

Equivalence Rules. We consider relaxing the inner-join ($\bowtie$) assumption to allow non-inner joins, such as left outerjoin ($⟕$), full outerjoin ($⟗$), left semijoin ($\ltimes$), and left antijoin ($\triangleright$). Rule 3 (UDF Reorder) still holds, while Rules 1 and 2 are analyzed below:

Table 1: Applicability of Rule 1 across different join operators $\odot \in \{\bowtie, ⟕, ⟗, \ltimes, \triangleright\}$ (✓: applicable; ✗: inapplicable; —: invalid).

$Req(U_k)$	Equivalence	$\bowtie$	⟕	⟗	$\ltimes$	$\triangleright$
$\{T_1\}$	$\sigma_{U_k}(T_1 \odot T_2) = \sigma_{U_k}(T_1) \odot T_2$	✓	✓	✗	✓	✓
$\{T_2\}$	$\sigma_{U_k}(T_1 \odot T_2) = T_1 \odot \sigma_{U_k}(T_2)$	✓	✗	✗	—	—

- **Rule 1 (UDF Pullup/Pushdown).** As shown in Table 1, beyond inner joins, Rule 1 is applicable only when $Req(U_k) = \{T_1\}$ and $\odot \in \{⟕, \ltimes, \triangleright\}$, while the remaining cases fall into two categories: (i) *Inapplicable:* the two sides of the equation are not equivalent. Take $\odot = ⟕$ with $Req(U_k) = \{T_2\}$ as an example, where $t_2 \in T_2$ is the only tuple matching $t_1 \in T_1$, and t_2 violates U_k. In $\sigma_{U_k}(T_1 ⟕ T_2)$, the join first produces (t_1, t_2), which is then filtered out by U_k; whereas in $T_1 ⟕ \sigma_{U_k}(T_2)$, t_2 is filtered out before the join, leaving t_1 with no matching tuple and thus outputting (t_1, NULL). The same holds for ⟗. (ii) *Invalid:* $\sigma_{U_k}(T_1 \odot T_2)$ cannot be evaluated. When $Req(U_k) = \{T_2\}$ and $\odot \in \{\ltimes, \triangleright\}$, $T_1 \odot T_2$ outputs only the attributes of T_1, and thus $\sigma_{U_k}(T_1 \odot T_2)$ lacks the attributes of T_2 required to evaluate U_k.
- **Rule 2 (Join Reorder).** For inner joins, commutativity and associativity are sufficient to cover all valid join reorderings. For non-inner joins, since ⟕, $\ltimes$, and $\triangleright$ are not commutative, some valid reorderings (e.g., $(T_1 \ltimes T_2) \ltimes T_3 = (T_1 \ltimes T_3) \ltimes T_2$) cannot be obtained from commutativity and associativity alone. To this end, Rule 2 can be extended to four rules: commutativity,

associativity, left asscom ($(T_1 \odot_1 T_2) \odot_2 T_3 = (T_1 \odot_2 T_3) \odot_1 T_2$), and right asscom ($T_1 \odot_1 (T_2 \odot_2 T_3) = T_2 \odot_2 (T_1 \odot_1 T_3)$). Prior work [5] has established the conditions under which these four rules hold across the above join operators, and each rule can be applied only when its conditions are satisfied.

Enumeration over the Augmented Join Graph. Based on the above analysis, when a query contains non-inner joins, we can jointly apply the extended Rules 1 and 2 along with the unchanged Rule 3 to its initial relational algebra expression. The resulting candidate plans constitute the UDF predicate placement space. However, performing a correct and complete enumeration over this space remains challenging. Existing enumeration methods (e.g., CD-C [5] and CD-E [1]) push down all predicates by default, and thus only enumerate the join ordering space under the extended Rule 2.

Nevertheless, the framework of CD-C [5] provides a promising basis for extending the enumeration over the augmented join graph to this space. Specifically, CD-C adapts inner-join enumeration algorithms (e.g., DPsize and DPsub) to non-inner joins by checking semantic constraints when enumerating each csg-cmp-pair (P_1, P_2). Following this idea, extending our enumeration requires designing new semantic constraints for csg-cmp-pair enumeration that jointly cover the extended Rules 1 and 2, as well as proving that the resulting enumeration preserves both soundness and completeness. Given the non-trivial theoretical effort required, we leave this extension to future work.

REFERENCES

[1] Altan Birler and Thomas Neumann. 2025. Efficient Enumeration of the Complete Join Search Space. *Proceedings of the 19th International Symposium on Database Programming Languages* (2025). https://api.semanticscholar.org/CorpusID:279586279

[2] Yannis Foufoulas, Theoni Palaiologou, and Alkis Simitsis. 2025. The UDFBench Benchmark for General-purpose UDF Queries. *Proceedings of the VLDB Endowment* 18, 9 (2025), 2804–2817.

[3] The Apache Software Foundation. 2025. Spark SQL PushPredicateThroughJoin Rule. https://github.com/apache/spark/blob/master/sql/catalyst/src/main/scala/org/apache/spark/sql/catalyst/optimizer/Optimizer.scala.

[4] Surabhi Gupta and Karthik Ramachandra. 2021. Procedural Extensions of SQL: Understanding their usage in the wild. *Proceedings of the VLDB Endowment* 14, 8 (2021), 1378–1391.

[5] Guido Moerkotte, Pit Fender, and Marius Eich. 2013. On the correct and complete enumeration of the core search space. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*. 493–504.